Sardar Vallabhbhai Patel Institute of Technology-Vasad

Subject: Software Project Management

Subject Code: 3171609

B.E. Sem-7

Project Name: Personal Expense Tracker

Team Members:

Enrollment No.	Name
220410116078	Meet Patel
220410116071	Jash Patel
220410116062	Brij Patel
220410116025	Harsh Shah

Practical-1

Aim: Prepare SRS for given software project.

Title: Personal Expense Tracker

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Intended Audience	3
1.3 Intended Use	3
1.4 Product Scope	3
1.5 Definitions and Acronyms	3
2. Overall Description	4
2.1 Product Perspective	4
2.2 Product Functions	4
2.3 User Classes and Characteristics	4
2.4 Operating Environment	4
2.5 Design and Implementation Constraints	4
2.6 Assumptions and Dependencies	4
3. External Interface Requirements	4
3.1 User Interface	4
3.2 Hardware Interfaces	4
3.3 Software Interfaces	5
3.4 Communication Interfaces	5
4. System Features/Functional Requirements	5
4.1 Expense Logging	5
4.2 Expense Report Generation	5

4.3 Notifications & Alerts 5

4.4 User Authentication 5

5. Non-Functional Requirements 5

6. Other Requirements 6

---

1. Introduction

1.1 Purpose The purpose of this document is to define the requirements for the Personal Expense Tracker (PET). The system automates the process of personal finance management using modern technologies such as receipt scanning with Optical Character Recognition (OCR). It aims to replace manual expense logging, reduce errors, save time, and provide real-time financial insights for users.

1.2 Intended Audience

- **Individuals** - to monitor and manage personal spending.
- **Students** - to track their budgets and expenses.
- **Professionals** - to generate spending reports, manage budgets, and track financial goals.
- **Project Developers & Testers** - to design, develop, and test the system.
- **Management** - to evaluate project progress and ensure compliance.

1.3 Intended Use

- Logging expenses quickly and easily when a transaction occurs.
- Viewing, editing, and categorizing expenses by the user.
- Providing daily, weekly, and monthly financial reports.
- Sending notifications for upcoming bills or for exceeding budget limits.
- Ensuring secure access with user-based login.

1.4 Product Scope The Personal Expense Tracker will:

- Automate expense logging through receipt scanning.
- Minimize manual intervention and data entry errors.
- Maintain a centralized and secure financial database.
- Support report generation for financial analysis and budgeting.
- Improve financial awareness and transparency for users.

1.5 Definitions and Acronyms

- **PET** - Personal Expense Tracker
- **OCR** - Optical Character Recognition
- **GUI** - Graphical User Interface
- **DBMS** - Database Management System

2. Overall Description

2.1 Product Perspective PET is an independent web/mobile-based system that connects with hardware (smartphone camera for OCR) and a centralized database. It interacts with users through a secure login-based interface.

2.2 Product Functions

- Automated expense logging via OCR.
- Real-time synchronization with the database across devices.
- Financial report generation.
- Notification system (e.g., email/push notification).
- Secure user access (authentication and profile management).

2.3 User Classes and Characteristics

- **Users:** Can add expenses, set budgets, view financial history and reports.

2.4 Operating Environment

- **Platform:** Web application (and optional Android/iOS app).
- **Database:** MySQL/PostgreSQL.
- **Server:** Apache / Node.js.
- **Hardware:** Smartphone camera for receipt scanning.

2.5 Design and Implementation Constraints

- Must comply with data privacy and financial security regulations.
- Secure login and encrypted data storage required.
- System should support a large number of users and transactions.
- Should be scalable to handle growing user data.

2.6 Assumptions and Dependencies

- Users must have a smartphone with a camera for receipt scanning.
- Internet connectivity required for synchronization.
- Users must review and validate data parsed from receipts by the system.

3. External Interface Requirements

3.1 User Interface

- Login/Registration screen.
- Dashboard for financial overview.
- Report generation panel.
- Notifications/alerts screen.

3.2 Hardware Interfaces

- Smartphone Camera for OCR.

- Server or Cloud database.

3.3 Software Interfaces

- **Database:** MySQL/PostgreSQL.
- **Notification APIs** (e.g., Twilio for SMS, SMTP for email).
- **OCR Service API** for receipt scanning.

3.4 Communication Interfaces

- Secure HTTPS communication.
- RESTful APIs for mobile/web integration.

4. System Features/Functional Requirements

4.1 Expense Logging

- **Description:** Log expenses manually or automatically by scanning a receipt.
- **Inputs:** Expense details (amount, category, date), receipt image.
- **Outputs:** Expense entry stored in the database.
- **Priority:** High.

4.2 Expense Report Generation

- **Description:** Generate daily/weekly/monthly spending reports.
- **Inputs:** Date range, category, payment method.
- **Outputs:** PDF/Excel reports.
- **Priority:** High.

4.3 Notifications & Alerts

- **Description:** Notify users about upcoming bill payments or when they exceed budget limits.
- **Inputs:** Budget limit, bill due date.
- **Outputs:** Notification sent.
- **Priority:** Medium.

4.4 User Authentication

- **Description:** Ensure secure access to personal financial data via login for each user.
- **Inputs:** Login credentials.
- **Outputs:** Access to the user's personal dashboard.
- **Priority:** High.

5. Non-Functional Requirements

- **Performance:** Must support 1000+ concurrent users.
- **Reliability:** System should have 99% uptime.
- **Security:** All data should be encrypted, secure login required.
- **Usability:** Easy-to-use GUI with minimal training.

- **Scalability:** Should scale to a growing number of users and transactions.

6. Other Requirements

- System should comply with financial data and privacy regulations.
- Backup and recovery features must be implemented.

Practical-2

Aim: Compare SDLC models for the given project.

1. Introduction Software Development Life Cycle (SDLC) models provide a structured approach to software development. Selecting the right SDLC model ensures that the project is delivered on time, within budget, and meets quality expectations. This document analyzes various SDLC models and justifies the selection of the most appropriate one for the Personal Expense Tracker.

2. Candidate SDLC Models We evaluated the following SDLC models for this project:

1. Waterfall Model
2. V-Model (Validation and Verification Model)
3. Incremental Model
4. Agile Model (Scrum-based)
5. Spiral Model

3. Chosen Model: Spiral Model The Spiral Model combines the iterative approach of prototyping with the systematic aspects of the Waterfall model. It emphasizes risk analysis, refinement, and user feedback in every cycle, which is perfect for a project like the Personal Expense Tracker.

Reasons of Spiral Model is Preferable for PET

1. **Risk Management**
   - Expense trackers may involve complex integrations with third-party services like OCR or banking APIs.
   - Spiral allows evaluating these technologies in early iterations, reducing the risk of choosing an unreliable API or service.

2. **Iterative Development with Feedback**
   - Instead of delivering the whole system at once, Spiral allows building in phases (e.g., Phase 1: Manual Entry, Phase 2: Reports, Phase 3: OCR Scanning).
   - Feedback from users can refine each phase to be more intuitive and useful.

3. **Flexibility in Requirements**
   - User financial needs often change (e.g., adding new expense categories, requesting new chart types).
   - Spiral supports requirement modifications better than Waterfall or V-Model.

4. **Scalability**
   - The Personal Expense Tracker may start with basic tracking and expand to include features like investment tracking or shared budgets.

- Spiral allows incremental scaling without redesigning the whole system.

5. **Early Prototyping**

- Spiral supports early prototypes (mockups of the expense entry form, interactive financial charts).
- This gives stakeholders (users) a chance to visualize the app before final implementation.

Comparison with Other Models

Model	Why Not Suitable for PET?
Waterfall	Too rigid; cannot handle frequently changing user financial needs (e.g., adding cryptocurrency tracking).
V-Model	Focuses on testing but still sequential, less flexible for changes.
Incremental	Good for partial delivery, but lacks structured risk analysis.
Agile	Flexible but may lack formal risk management, which is critical due to dependencies on third-party APIs (OCR, financial data).

Conclusion: After evaluating multiple SDLC models, the Spiral Model emerges as the most suitable choice for developing the Personal Expense Tracker (PET). Unlike rigid models such as Waterfall and V-Model, or flexible but less risk-focused models like Agile and Incremental, the Spiral Model balances structured development with iterative prototyping and continuous risk analysis. Its ability to adapt to changing requirements, manage risks effectively, and support early prototyping makes it ideal for a project like PET, which involves API integration (OCR), evolving user needs, and scalability of features. Therefore, the Spiral Model ensures that PET can be developed efficiently, flexibly, and with reduced project risks, ultimately delivering a reliable and future-ready solution.

Practical-3

Aim: Estimate project cost and prepare project schedule.

1. Introduction Project cost estimation and scheduling are critical activities in software project management. For the Personal Expense Tracker, this practical outlines the estimated effort, cost, and a detailed timeline required for its development. We will use the Intermediate COCOMO (Constructive Cost Model) for effort and duration estimation, as it provides a more accurate forecast by considering project-specific attributes. Following the estimation, a detailed project schedule will be prepared using a Work Breakdown Structure (WBS) and presented as a Gantt Chart.

2. Project Cost Estimation (Using Intermediate COCOMO Model) The Intermediate COCOMO model provides a more accurate estimate than the Basic model by considering 15 "cost drivers" that affect project productivity. We will analyze the most relevant drivers for the Personal Expense Tracker to calculate an Effort Adjustment Factor (EAF).

2.1. Estimating Project Size The project size estimate is based on the features defined in the SRS (Software Requirements Specification).

- **Estimated KLOC:** 10 KLOC (10,000 Lines of Code)

2.2. Cost Drivers and Effort Adjustment Factor (EAF) Based on the project's characteristics, we've assigned ratings to key cost drivers. The project is being built by a small, skilled team using modern tools, which results in an EAF value less than 1.0, indicating higher-than-average productivity.

Cost Driver	Description	Rating	Value	Justification
RELY	Required Software Reliability	High	0.82	This system handles critical personal financial data; high reliability is required to avoid errors or data loss.
CPLX	Product Complexity	High	1.15	The project involves complex OCR for receipt scanning and potential integration with third-party financial APIs.
PCAP	Programmer Capability	High	0.86	We assume the student team is skilled in the required technologies (e.g., Python, OCR libraries, web development).
VEXP	Virtual Machine Experience	High	0.90	The team is familiar with the runtime environments for the chosen technologies.
MODP	Modern Programming Practices	High	0.91	The project uses modern practices like Agile, microservices architecture, and RESTful APIs.

The Effort Adjustment Factor (EAF) is the product of all driver values:
EAF=0.82×1.15×0.86×0.90×0.91 EAF≈0.664

2.3. Intermediate COCOMO Calculations We will use the formulas for the Organic Mode with the calculated EAF.

- Effort = 2.4×(KLOC)1.05×EAF Person-Months
- Time (TDEV) = 2.5×(Effort)0.38 Months
- Staff Required = Effort / TDEV Persons

Calculations:

1. **Effort Calculation:** Effort = 2.4×(10)1.05×0.664 Effort ≈2.4×11.22×0.664 Effort ≈16.733 Person-Months
2. **Time (Duration) Calculation:** TDEV = 2.5×(16.733)0.38 TDEV ≈2.5×2.92 TDEV ≈7.293 Months (Approximately 32 weeks)
3. **Staffing Calculation:** Staff = 16.733/7.293 Staff ≈2.29 Persons (This confirms a core team of 2-3 developers is ideal)

2.4. Cost Estimation

- **Assumed Average Developer Salary:** ₹40,000 per month
- **Total Effort:** 16.733 Person-Months
- **Total Development Cost:** Effort (PM) × Salary per Month
  - Cost = 16.733×40,000
  - **Total Estimated Cost = ₹6,69,320**

3. Project Schedule The project schedule is broken down into phases and tasks based on the Agile methodology. The total duration aligns with our COCOMO estimate of ~8 months.

3.1. Work Breakdown Structure (WBS)

1. **Phase 1: Project Planning & Design (Weeks 1-5)**
   - 1.1. Requirement Analysis Finalization
   - 1.2. System Architecture Design
   - 1.3. Database Schema Design (for user transactions, budgets)
   - 1.4. UI/UX Wireframing
2. **Phase 2: Sprint 1 - Core Functionality (Weeks 6-12)**
   - 2.1. Setup Development Environment
   - 2.2. User Registration & Profile Management
   - 2.3. Manual Expense & Income Entry Module
   - 2.4. API for CRUD on Transactions
3. **Phase 3: Sprint 2 - Budgeting & Dashboard (Weeks 13-22)**
   - 3.1. Budget Creation & Management Module
   - 3.2. Interactive Financial Dashboard (Charts & Summaries)

- 3.3. User Dashboard for Viewing Expense History
- 3.4. Unit & Integration Testing for Sprint 1 & 2

4. **Phase 4: Sprint 3 - Reporting & Advanced Features (Weeks 23-30)**

   - 4.1. OCR Integration for Receipt Scanning
   - 4.2. APIs for Managing Categories & Budgets
   - 4.3. Custom Report Generation (PDF/CSV Export)
   - 4.4. Bill Reminder & Notification System

5. **Phase 5: Testing, Deployment & Documentation (Weeks 31-32)**

   - 5.1. End-to-End System Testing
   - 5.2. Performance & Security Testing
   - 5.3. Deployment to Production Server
   - 5.4. User Documentation & Final Report Preparation

3.2. Gantt Chart Schedule

Sr. no.	Phase / Milestone	Start Week	End Week	Duration (Weeks)
1	Phase 1: Project Planning & Design	1	5	5
2	Phase 2: Sprint 1 - Core Functionality	6	12	7
3	Phase 3: Sprint 2 - Budgeting & Dashboard	13	22	10
4	Phase 4: Sprint 3 - Reporting & Advanced Features	23	30	8
5	Phase 5: Testing & Deployment	31	32	2

4. Conclusion Based on the Intermediate COCOMO model, the Personal Expense Tracker is estimated to require approximately 16.733 Person-Months of effort over a duration of about 8 months, with a development cost of roughly ₹6,69,320. The project schedule, detailed in the Gantt chart, provides a clear roadmap for executing the project in phases. This structured approach, aligned with the Agile methodology, ensures timely delivery and flexibility throughout the development lifecycle.

Practical-4

Aim: Evaluate risk management approaches suitable for the project.

Risk Management for PET Risk management in software project management involves identifying, analyzing, and mitigating risks that could affect the project's success.

Risk Identification (Possible Risks in PET)

1. **Technical Risks**
    - Third-party API (OCR/Plaid) failure or unexpected changes.
    - Integration issues between APIs and the core application.
    - Database failure or data loss.
2. **Project Management Risks**
    - Schedule delays due to changing user requirements.
    - Underestimation of project complexity (effort/cost).
    - Limited skilled manpower for secure API integration and data protection.
3. **Operational Risks**
    - Internet outages preventing data sync across devices.
    - User error in manual data entry leading to incorrect financial reports.
4. **Security Risks**
    - Unauthorized access to sensitive financial records.
    - Data privacy violations (user information leakage).
5. **External Risks**
    - Change in financial data regulations (e.g., GDPR).
    - Vendor dependency for API services.

Risk Management Approaches

1. **Risk Avoidance**
    - Choose reliable, well-documented, and stable third-party APIs.
    - Use well-established technologies (e.g., MySQL, secure APIs).
    - Define clear requirements and scope with stakeholders before starting development.
2. **Risk Mitigation**
    - Maintain backup options (manual entry and correction if OCR fails or is inaccurate).

- Provide data redundancy and automatic database backups.
- Create clear user guides and tutorials to minimize data entry errors.

3. **Risk Transfer**

- Outsource SMS/Email notifications to reliable third-party APIs (e.g., Twilio).
- Use a paid, reliable OCR service with a Service Level Agreement (SLA).

4. **Risk Acceptance**

- Minor UI/UX issues may be accepted initially and improved in later releases.
- Low-impact risks (like slight reporting delays) may be tolerated.

Risk Assessment Matrix (FMEA Approach)

Risk	Impact (1–10)	Probability (1–10)	Risk Priority Number (RPN = I × P)	Action
Third-party API Unavailability	8	6	48	Use services with high uptime, have a fallback plan.
Internet outage	7	5	35	Local offline storage + sync later.
Data breach	10	4	40	Encrypt all data, strong access control, regular security audits.
Incorrect OCR Data Parsing	7	5	35	Allow easy manual correction of scanned data.
Schedule delay	7	5	35	Use Spiral model + buffer time.
Requirement changes	6	6	36	Iterative prototyping + feedback.
User Adoption/Trust Issues	8	4	32	Provide clear privacy policy, training sessions, and be transparent about data usage.

Conclusion: Effective risk management is essential for the successful implementation of the Personal Expense Tracker (PET). By identifying risks across technical, project management, operational, security, and external domains, the project team can proactively plan mitigation strategies. The adoption of avoidance, mitigation, transfer, and acceptance approaches, supported by an FMEA-based risk assessment matrix, ensures that high-priority risks such as API failures, data breaches, and schedule delays are controlled effectively. Among the SDLC models, the Spiral Model complements this approach best, as it emphasizes continuous risk analysis, early prototyping, and iterative feedback. With this structured risk management plan, PET can be developed in a way that minimizes uncertainties, improves reliability, and ensures long-term success and user trust.